Sanaul Haque

Ye_Moe_Yan_Lin_Proposal

Course 608

Document Details

Submission ID
trn:oid:::3117:637083214

Submission Date
Oct 10, 2026, 11:50 PM GMT+6

Download Date
Oct 10, 2026, 11:57 PM GMT+6

File Name
Ye_Moe_Yan_Lin_Proposal.pdf

File Size
299.7 KB

7 Pages

1,939 Words

12,325 Characters

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

16 Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

4 Missing Citation 4%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

12% Internet sources
13% Publications
15% Submitted works (Student Papers)

Match Groups

- **16** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **4** Missing Citation 4%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

12% Internet sources
13% Publications
15% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		4%
2	Publication	
Suresh Babu Nettur, Shanthi Karpurapu, Unnati Nettur, Likhit Sagar Gajja. "Cypre...		2%
3	Internet	
trepo.tuni.fi		2%
4	Publication	
Kyori Tandiah, Yohannes Kurniawan. "Trust and Performance Outweigh Privacy: ...		1%
5	Student papers	
Monash University on 2026-10-03		1%
6	Student papers	
Victorian Institute of Technology on 2026-08-21		1%
7	Internet	
journal-isi.org		1%
8	Publication	
Elizabeth Seger, Aviv Ovadya, Divya Siddarth, Ben Garfinkel, Allan Dafoe. "Democ...		1%
9	Student papers	
University of Dundee on 2025-04-23		<1%
10	Student papers	
Victorian Institute of Technology on 2026-10-09		<1%

11 Student papers

George Mason University on 2026-10-03 <1%

12 Publication

Yu Jiang, Meixin Qu, Feng Cheng, Huifen Sun, Jie Dai, Zhen Wu. "Correlation of tel... <1%

13 Student papers

North Shore International Academy on 2025-07-20 <1%

14 Student papers

ESLSCA Business School (Cairo) on 2026-05-16 <1%

15 Student papers

Miami University of Ohio on 2025-11-10 <1%

16 Student papers

University of Salford on 2025-01-05 <1%

17 Internet

www.frontiersin.org <1%

INTERNATIONAL ACADEMIC RESEARCH COMPETITION

RESEARCH PROPOSAL ON

16

GENERATIVE ARTIFICIAL INTELLIGENCE IN SOFTWARE DEVELOPMENT: ITS EFFECTS ON PRODUCTIVITY, CODE QUALITY, AND LEARNING AMONG STUDENT AND EARLY-CAREER DEVELOPERS

INSTITUTION: IIP International School

CATEGORY: Junior

COUNTRY: Myanmar

SCHOOL YEAR: IGCSE

PROPONENT: Ye Moe Yan Lin

DATE OF SUBMISSION: 20/9/2026

Title: The influence of Generative Artificial Intelligence on Software-Development Productivity and Code Quality in Novice and Student Developers

1 Introduction

Software development is increasingly influenced by Generative Artificial Intelligence (GenAI), particularly large language model tools such as ChatGPT and GitHub Copilot. These systems can generate code, explain programming concepts, suggest debugging steps, create tests, and provide alternative solutions. For university students and early-career developers, GenAI may reduce the difficulty of unfamiliar programming tasks and help users complete software activities more quickly [2], [3], [4]. Evidence from controlled and field studies indicates that coding assistants can improve task progress and efficiency, although the size of the benefit varies according to task type, user experience, and the way the tool is used [6], [8], [9].

Despite these benefits, faster code production does not necessarily produce better software. AI-generated suggestions may contain security weaknesses, coding-rule violations, unnecessary complexity, or maintenance problems when users accept them without adequate testing and review [1], [5], [7]. Students may also become dependent on generated solutions and develop less opportunity to practise independent problem-solving and code explanation [3], [4]. These concerns show that the influence of GenAI should be evaluated through both productivity and code quality, while also considering the learner's verification behaviour and retained understanding.

To address this issue, this study will develop a controlled comparison between GenAI-assisted and manual programming. It will examine whether GenAI improves programming performance, whether it affects measurable code quality, and whether responsible use depends on the user's ability to review and understand generated code.

2 Research Questions

This study aims to determine the effects of GenAI-assisted programming on the performance, code quality, and learning-related behaviour of university students and early-career developers. Specifically, this study intends to answer the following questions:

1. How effective is GenAI-assisted programming in terms of: a. speed of completing programming tasks; b. number of successfully completed requirements; c. debugging and testing performance; and d. perceived productivity?
2. How effective is the code produced by GenAI-assisted and manual-coding participants in terms of: a. security weaknesses; b. cyclomatic and cognitive complexity; c. maintainability; and d. coding-rule violations?

12

3. Is there a significant difference between the GenAI-assisted group and the manual-coding group in terms of task completion time, task success, and code quality?
4. What is the relationship between the participant's level of AI reliance and their verification behaviour, confidence in the produced code, and performance in a short no-AI programming assessment?
5. What is the acceptability of GenAI-assisted programming in terms of usefulness, ease of use, trust, and responsible adoption?

3 Literature Review

GenAI coding assistants have introduced a new form of human–computer collaboration in software development. These systems can provide code completion, explanations, examples, and debugging suggestions, which may help students overcome syntax problems and begin unfamiliar tasks more confidently [2], [3]. Research involving computing students has reported improvements in programming effectiveness and efficiency when GitHub Copilot is available, particularly in tasks that require work inside an existing codebase [6]. Field evidence also suggests that GenAI can increase developer productivity, but the effect is not uniform across all users and tasks [8], [9].

The benefits of speed must be considered alongside software quality. A study of AI-assisted programming found that users could produce code with more security vulnerabilities and integer-overflow mistakes, demonstrating that generated code still requires human testing and review [5]. Other work has examined static-analysis warnings, complexity, code smells, and maintainability in AI-supported development [1], [7]. These studies indicate that completion time alone is not a sufficient measure of successful software development because code that is produced quickly may require additional correction or create future technical debt.

The educational effects of GenAI are equally important. Students report that AI code-completion tools can help them understand unfamiliar syntax, generate ideas, and solve programming problems, but they also identify inaccurate suggestions, reduced independent thinking, and uncertainty about whether generated code is correct [3]. This creates a need to distinguish useful assistance from uncritical dependence. Human expertise remains necessary for interpreting requirements, selecting appropriate solutions, checking security, and explaining why a program works [4], [7].

13

Taken together, the literature supports a comparative study that measures productivity and code quality at the same time. It also supports the inclusion of a no-AI assessment and questions about verification, because a tool may improve immediate task performance while producing weaker independent understanding. This study will therefore focus on the conditions

under which GenAI becomes a useful programming aid rather than a replacement for developer judgement.

4 Proposed Methodology: The method is a quantitative approach

1. Participant Selection: Approximately 60 university students and early-career developers with basic programming experience will participate in the study. Participants will be grouped according to comparable programming experience and then assigned to either a GenAI-assisted group or a manual-coding group. Participation will be voluntary, and all personal information, code submissions, and survey responses will be anonymized.

2. Task Preparation: The participants will complete equivalent programming tasks involving feature implementation, debugging, and testing. The tasks will include both a new-project activity and an activity inside an existing codebase. The programming language, task instructions, time limit, reference materials, testing requirements, and evaluation criteria will be standardized. The GenAI-assisted group will use one approved coding assistant under defined rules, while the manual-coding group will use ordinary documentation and search resources without generative code completion.

3. Productivity Evaluation: The productivity of each participant will be evaluated using task-completion time, successfully implemented requirements, passing test cases, debugging progress, and the number of revisions required before submission. A short questionnaire will also measure perceived productivity, confidence, and difficulty. These measures will allow the study to compare both observable performance and the participants' experience of using the technology.

4. Code-Quality Evaluation: Every submitted program will be tested and analysed using the same evaluation procedure. Automated static-analysis tools will be used to identify security weaknesses, coding-rule violations, cyclomatic complexity, cognitive complexity, and maintainability indicators. A second review of selected submissions will be conducted using a common rubric to examine correctness, readability, and unnecessary code. The combination of automated and rubric-based evaluation will reduce dependence on a single quality measure [1], [5], [7].

15

5. Data Analysis and Acceptability Rating: Descriptive statistics, independent-samples tests, and regression analysis will be used to examine differences between the two groups and the possible effects of prior experience and task complexity. The participant's AI reliance will be estimated from usage records and survey responses, while verification behaviour will be measured through questions about testing, revising, and explaining generated code. After the programming tasks, participants will complete a short no-AI explanation or debugging activity.

Acceptability will be assessed using a four-point Likert-scale questionnaire covering usefulness, ease of use, trust, and responsible adoption.

5 Expected Outcome

The study is expected to demonstrate that GenAI-assisted programming can improve task-completion speed and initial progress, particularly when participants are working with unfamiliar code or requirements [6], [8], [9]. However, the results may also show that productivity gains differ according to programming experience, task complexity, and the amount of human verification performed.

The study is further expected to identify differences in code quality between AI-assisted and manual submissions. AI-assisted code may contain more security, complexity, or maintainability concerns when suggestions are accepted without adequate testing [1], [5], [7]. The no-AI assessment may show that participants who actively review and explain generated code retain stronger understanding than participants who rely on the tool without verification. The results will be used to propose practical guidelines for responsible GenAI use in programming classes and early-career software work.

6 Potential Limitation

The study may be limited by the sample size, the participants' unequal familiarity with GenAI tools, and the restricted number of programming tasks. Results may also vary according to the selected AI system, programming language, prompt quality, and difficulty of the software projects. Static-analysis tools cannot measure every aspect of real-world maintainability, security, or user experience. These limitations will be addressed through standardized tasks, transparent usage rules, multiple evaluation measures, and clear reporting of participant experience.

7 Conclusion

GenAI is changing software development by influencing how programmers write, debug, test, and learn. Its ability to generate code can provide meaningful productivity benefits, but its suggestions still require human judgement because speed does not guarantee security, maintainability, or understanding. By comparing GenAI-assisted programming with manual coding, this study will determine how the technology affects students and early-career developers across performance, code quality, verification, and learning-related outcomes.

The proposed research is important because it does not treat GenAI as either completely beneficial or completely harmful. Instead, it aims to identify the conditions under which the technology can support programming education and software development without weakening code-review practices and independent technical reasoning.

8 Project Practicalities

The project will be conducted over seven months. The first month will cover the finalization of the research protocol, preparation of the programming tasks, development of the evaluation rubric, and participant recruitment. The second and third months will be used for pilot testing and the controlled programming sessions. The fourth month will focus on code collection, testing, static analysis, and completion of the participant questionnaires. The fifth and sixth months will be dedicated to statistical analysis and interpretation of the productivity, quality, reliance, and learning results. The final month will be used to prepare the recommendations, complete the research paper, and present the findings.

9 Post-program Plan

After the study, the researchers intend to improve the instruments and repeat the experiment with a larger and more diverse sample. The findings may be presented to computing teachers, student research organizations, and software-development communities. The proposed guidelines may also support future studies on AI literacy, secure code review, programming education, and responsible adoption of coding assistants in schools and early-career workplaces.

REFERENCES

[1] P. Haindl and G. Weinberger, “Does ChatGPT Help Novice Programmers Write Better Code? Results from Static Code Analysis,” 2024. DOI: 10.20944/preprints202406.1151.v1.

[2] X. Tan, X. Long, X. Ni, Y. Zhu, J. Jiang, and L. Zhang, “How far are AI-powered programming assistants from meeting developers’ needs?” *arXiv.org*, 2024. DOI: 10.48550/arxiv.2404.12000.

[3] W. Takerngsaksiri, C. Warusavitarne, C. Yaacoub, M. H. K. Hou, and C. Tantithamthavorn, “Students’ Perspective on AI Code Completion: Benefits and Challenges,” *arXiv.org*, 2023. DOI: 10.48550/arxiv.2311.00177.

[4] H. Sandhaus, Q. Gu, M. T. Parreira, and W. Ju, “Student Reflections on Self-Initiated GenAI Use in HCI Education,” 2024. DOI: 10.48550/arxiv.2410.14048.

[5] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?” *arXiv.org*, 2022. DOI: 10.48550/arXiv.2211.03622.

[6] M. I. H. Shihab, C. Hundhausen, A. Tariq, S. Haque, and B. Mulanda, “The Effects of GitHub Copilot on Computing Students’ Programming Effectiveness, Efficiency, and Processes in Brownfield Programming Tasks,” 2025. DOI: 10.1145/3702652.3744219.

[7] N. Pombo and C. Alves e Cunha, “BRIDGING Human Expertise and Generative AI in Software Quality: A Study on Code Smell Detection in Software Engineering Education,” 2025. DOI: 10.1109/tale66047.2025.11346609.

[8] K. Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz, “The Productivity Effects of Generative AI: Evidence from a Field Experiment with GitHub Copilot,” 2024. DOI: 10.21428/e4baedd9.3ad85f1c.

[9] D. Kreitmeir and P. A. Raschky, “The Heterogeneous Productivity Effects of Generative AI,” *Social Science Research Network*, 2024. DOI: 10.2139/ssrn.4745624.

[10] M. Borg, D. W. Hewett, D. R. Graham, N. Couderc, E. Söderberg, L. Church, and D. Farley, “Does Co-Development with AI Assistants Lead to More Maintainable Code? A Registered Report,” 2024. DOI: 10.48550/arxiv.2408.10758.